

Class Project

AN OVERVIEW OF LLM SERVICE INTEGRATION WITH JAVA APPLICATION

BITP 3123 Distributed Application Development

Author

Emaliana Kasmuri

Fakulti Teknologi Maklumat dan Komunikasi

Universiti Teknikal Malaysia Melaka

Table of Contents

1	Document History	1
2	Description	1
3	Step 1 of 4: Ollama Installation & LLM Pull.....	2
3.1	Install Ollama.....	2
3.1.1	macOS	2
3.1.2	Windows.....	2
3.2	Start the Ollama server	2
3.2.1	macOS	2
3.2.2	Windows.....	3
3.3	Pull LLMs	3
3.4	Verify Pulled Model.....	4
4	Step 2 of 4: Project Configuration	5
4.1	Create the Maven Project in Eclipse	5
4.2	Open pom.xml in raw XML view	6
4.3	Replace the pom.xml content.....	6
4.4	Update Maven Project in Eclipse.....	1
4.5	Verify dependencies resolved.....	1
5	Step 3 of 4: Java Implementation	1
5.1	Create the package	1
5.2	Create the LLMSERVICE class	1
5.3	Create the verification runner class	9
5.4	Ensure Ollama is running	14
5.5	Run the test	14
6	Summary of Implementation	14

1 Document History

Date	Description
5 April 2026	Document creation.
18 June 2026	Change LLM size from large to small-to-medium size. Section 3.3, 3.4, 5.2 and 5.3.

2 Description

This section describes the integration of locally hosted Large Language Models (LLMs) into a Java-based application using Ollama as the model serving platform. Ollama functions as a lightweight local inference server that runs as a background process on the developer's machine and exposes an HTTP API endpoint at localhost:11434. This architecture allows the Java application to interact with multiple LLMs without requiring external cloud services, thereby ensuring offline capability, reduced latency, and better control over data privacy.

The implementation utilises five open-source language models hosted within Ollama. These include **Llama 3.2 (3B)** as a general-purpose multilingual baseline, **Phi-4-mini (3.8B)** for efficient inference and lightweight processing, **Qwen 2.5 (3B)** for strong bilingual and code-switching capabilities, **Gemma-SEA-LION v4 (4B)** for Southeast Asian regional adaptation with strong Malay support, and **MedGemma (4B)** for domain-specific biomedical and nutritional knowledge. These models are individually pulled from the Ollama repository and run locally, enabling the system to support diverse analytical tasks ranging from general query processing to specialised nutritional analysis.

Integration with the Java application is achieved through the **LangChain4j** framework, specifically using the Ollama integration module defined in the Maven configuration. The dependency setup allows the Java application to communicate seamlessly with the Ollama server without requiring low-level HTTP handling.

The core of this integration is implemented through the **LLMService** class, which acts as an abstraction layer between the application logic and the underlying LLMs. This class establishes a connection to the local Ollama server by specifying the base URL and dynamically selecting the desired model. A reusable method is provided to send prompts to any of the registered models and retrieve responses in real time. This design promotes modularity and allows other components of the system, such as the nutritional analysis pipeline, to interact with different models using a unified interface.

To validate the integration, a standalone Java runner class is implemented to test the connectivity between the Java application and each LLM. The test process confirms that all models are correctly loaded in Ollama and can generate responses. This ensures that the end-to-end pipeline—from model hosting to application-level interaction—is functioning correctly before further system development is carried out.

Overall, this section establishes a complete local LLM integration architecture, where Ollama serves as the model provider and Java (via LangChain4j) acts as the client application. This approach supports scalable experimentation with multiple models while maintaining a consistent and reusable integration structure.

3 Step 1 of 4: Ollama Installation & LLM Pull

Purpose

Ollama is the local model server that your Java program will talk to. It runs as a background process on each developer's machine and exposes a simple HTTP API on localhost:11434. Without this, there is nothing for LangChain4j to connect to.

Expected outcome

By the end of this step all 4 models are downloaded and responding to prompts from the Terminal/Command Prompt. The student will be able to confirm each model works before writing a single line of Java.

3.1 Install Ollama

3.1.1 macOS

1. Go to <https://ollama.com/download/mac>
2. Download the **.dmg** file.
3. Open it and drag **Ollama** to the **Applications** folder.

3.1.2 Windows

1. Download and run the installer from <https://ollama.com/download/windows>. It installs **Ollama** as a background service automatically.

3.2 Start the Ollama server

3.2.1 macOS

1. Open **Ollama** from **Applications**. The Ollama notification shall appear as shown in Figure 1.

Phi-4-mini (3.8B)	ollama pull phi4-mini
MedGemma 4B	ollama pull medgemma:4b
Gemma-SEA-LION v4 4B	ollama pull aisingapore/Gemma-SEA-LION-v4-4B-VL

3.4 Verify Pulled Model

1. Test each model with a quick prompt as shown in Figure 4, in a separate Terminal or Command Prompt session.

```
ollama run llama3.2:3b "Name three common cooking ingredients."
```

Figure 4: Quick prompt test

The output shall be similar as shown in Figure 5.

```

emalianakasmuri ~ - zsh -- 152x42
Last login: Thu Jun 18 08:20:43 on ttys004
emalianakasmuri@Emalias-MacBook-Pro ~ % ollama pull llama3.2:3b
pulling manifest
pulling dde5aa3fc5ff: 100% ██████████ 2.0 GB
pulling 966de95ca8a6: 100% ██████████ 1.4 KB
pulling fec5a6bec9da: 100% ██████████ 7.7 KB
pulling a70ff7e570d9: 100% ██████████ 6.0 KB
pulling 56bb8bd477a5: 100% ██████████ 96 B
pulling 34bb5ab01051: 100% ██████████ 561 B
verifying sha256 digest
writing manifest
success
emalianakasmuri@Emalias-MacBook-Pro ~ % ollama run llama3.2:3b "Name three common cooking ingredients."
Here are three common cooking ingredients:
1. Salt
2. Flour
3. Sugar
emalianakasmuri@Emalias-MacBook-Pro ~ %

```

Figure 5: Output from the quick prompt

2. Repeat step 1 using the following describe in Table 1, running one at a time or simultaneously in different tab sessions.

Table 2: LLM and command to test the pulled model

Model	Run command
Qwen 2.5 3B	ollama run qwen2.5:3b "Name three common cooking ingredients."
Phi-4-mini (3.8B)	ollama run phi4-mini "Name three common cooking ingredients."
MedGemma 4B	ollama run medgemma:4b "Name three common cooking ingredients."
Gemma-SEA-LION v4 4B	ollama run aisingapore/Gemma-SEA-LION-v4-4B-VL "Name three common cooking ingredients."

- Each should return a short coherent response. If any model fails, re-run the pull command for that model.

4 Step 2 of 4: Project Configuration

Purpose

Set up the **Maven** build file with the correct **Java** version, **LangChain4j** core, and **Ollama** integration dependencies.

Expected outcome

Eclipse resolves all dependencies without red error markers under **Maven Dependencies** in the **Package Explorer**.

4.1 Create the Maven Project in Eclipse

- Go to **File > New > Maven Project**
- Tick **Create a simple project (skip archetype selection)** → click **Next**
- Fill in the coordinates using the information in Table 3

Table 3: Maven coordinates for the project

Field	Value
Group ID	edu.utem.ftmk

Artifact ID	nutritional-llm-service
Version	1.0.0
Packaging	jar

4. Click **Finish**. The project should appear in the **Package Explorer** with a **pom.xml** inside.

4.2 Open pom.xml in raw XML view

1. Expand the project in **Package Explorer**.
2. Double-click **pom.xml**
3. Click the **pom.xml** tab at the bottom of the editor to switch to raw XML view.

4.3 Replace the pom.xml content

1. Select all (Ctrl+A) and replace with the code shown in Figure 1.


```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>edu.utem.ftmk</groupId>
  <artifactId>nutritional-llm-service</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>
  <name>Nutritional LLM Service</name>

  <properties>
    <java.version>11</java.version>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <langchain4j.version>1.11.0</langchain4j.version>
```

```
</properties>
```

```
<dependencies>
```

```
<!-- LangChain4j core -->
```

```
<dependency>
```

```
  <groupId>dev.langchain4j</groupId>
```

```
  <artifactId>langchain4j</artifactId>
```

```
  <version>${langchain4j.version}</version>
```

```
</dependency>
```

```
<!-- Ollama integration -->
```

```
<dependency>
```

```
  <groupId>dev.langchain4j</groupId>
```

```
  <artifactId>langchain4j-ollama</artifactId>
```

```
  <version>${langchain4j.version}</version>
```

```
</dependency>
```

```
<!-- Logging -->
```

```
<dependency>
```

```
<groupId>org.slf4j</groupId>  
<artifactId>slf4j-simple</artifactId>  
<version>2.0.13</version>  
</dependency>  
  
</dependencies>  
  
</project>
```

Figure 6: pom.xml dependencies definition for the project

2. Save the file.

4.4 Update Maven Project in Eclipse

1. Right-click the project in Package Explorer
2. Select **Maven → Update Project**
3. Tick **Force Update of Snapshots/Releases**
4. Click **OK**. Wait for the progress bar at the bottom right of Eclipse to clear completely before proceeding.

4.5 Verify dependencies resolved

1. Expand **Maven Dependencies** under your project in Package Explorer. Confirm these three JARs are present with no red markers:
 - a. langchain4j-1.11.0.jar
 - b. langchain4j-ollama-1.11.0.jar
 - c. slf4j-simple-2.0.13.jar

5 Step 3 of 4: Java Implementation

Purpose

Create a reusable Java class that connects to the local Ollama server and sends prompts to any of the 4 models. This is the core integration class your nutritional analysis pipeline will call.

Expected outcome

A working Java class that you can run directly in Eclipse. It will send a simple test prompt to each of the 4 models and print the responses to the console, confirming the Java-to-Ollama connection works.

5.1 Create the package

1. Expand nutritional-llm-service → right-click src/main/java.
2. Select New → Package
3. Enter edu.utem.ftmk.llm → click Finish

5.2 Create the LLMService class

1. Right-click the edu.utem.ftmk.llm package → **New → Class**

2. Name it LLMSERVICE → click **Finish**
3. Replace the entire content the code shown in Figure 7.
4. Save the class.


```
package edu.utem.ftmk.llm;

import dev.langchain4j.model.chat.ChatModel;
import dev.langchain4j.model.ollama.OllamaChatModel;

import java.time.Duration;

/**
 * LLMService provides a reusable interface for sending prompts to locally
 * hosted Large Language Models (LLMs) via Ollama.
 *
 * Ollama runs as a background server on the developer's machine and exposes
 * a REST API on localhost:11434. This class uses LangChain4j's OllamaChatModel
 * to communicate with that server.
 *
 * Supported models:
 * - Llama 3.2 3B : General-purpose multilingual baseline
```

- * - Phi-4-mini 3.8B : Efficient inference, good for batch processing
- * - Qwen 2.5 3B : General multilingual, Malay-English code-switching
- * - Gemma-SEA-LION 4B : Southeast Asian regional model, strong Malay
- * - MedGemma 4B : Domain-specific biomedical and nutritional knowledge

*

* Usage example:

```
* LLMService service = new LLMService();
* String response = service.prompt(LLMService.LLAMA, "Your prompt here");
*/
```

```
public class LLMService {
```

```
/**
```

- * The base URL where the Ollama server is running.
- * This is the default address when Ollama is installed locally.
- * Change this if Ollama is running on a different host or port.

```
*/
```

```
private static final String OLLAMA_BASE_URL = "http://localhost:11434";
```



```

/**
 * Model name constants for the four LLMs used in this study.
 * These strings must match exactly the model names registered in Ollama.
 * Use these constants instead of hardcoding model names in other classes.
 */

public static final String LLAMA  = "llama3.2:3b";
public static final String PHI    = "phi4-mini";
public static final String QWEN   = "qwen2.5:3b";
public static final String SEALION = "aisingapore/Gemma-SEA-LION-v4-4B-VL";
public static final String MEDGEMMA = "medgemma:4b";

/**
 * Builds and returns a ChatModel instance connected to the specified
 * Ollama model. The model is configured with a 5-minute timeout to
 * accommodate slower CPU-based inference on local machines.
 *

```

```

* @param modelName The Ollama model name (use the constants above)
* @return      A ChatModel ready to accept prompts
*/
public ChatModel buildModel(String modelName) {
    return OllamaChatModel.builder()
        .baseUrl(OLLAMA_BASE_URL) // Connect to local Ollama server
        .modelName(modelName)      // Select which LLM to use
        .timeout(Duration.ofMinutes(5)) // Allow up to 5 min for response
        .build();
}

/**
* Sends a text prompt to the specified LLM and returns the response
* as a plain string.
*
* This is the main method other classes will call to interact with
* any of the four models in this study.

```

```

*
* @param modelName The Ollama model name (use the constants above)
* @param userPrompt The prompt text to send to the model
* @return          The model's response as a plain string
*/
public String prompt(String modelName, String userPrompt) {
    ChatModel model = buildModel(modelName); // Initialise the chosen model
    return model.chat(userPrompt);          // Send prompt and return response
}
}

```

Figure 7: LLMService.java (updated)

5.3 Create the verification runner class

1. Right-click the edu.utem.ftmk.llm package → **New → Class**
2. Name it LLMServiceTest → tick **public static void main** → click **Finish**
3. Replace the entire content the code shown in Figure 8.
4. Save the class.

```
package edu.utem.ftmk.llm;

/**
 * LLMSERVICE is a simple verification runner that confirms all four
 * locally hosted LLMs are reachable and responding correctly via LLMSERVICE.
 *
 * This is not a unit test framework class (no JUnit). It is a standalone
 * Java application intended to be run manually in Eclipse to verify the
 * Ollama server is running and all models are loaded correctly before
 * beginning the nutritional analysis experiments.
 *
 * How to run:
 * 1. Ensure Ollama is running (look for the Ollama icon in the menu bar)
 * 2. Right-click this file in Eclipse -> Run As -> Java Application
 * 3. Check the Console tab for responses from each model
 *
 * Expected output:
 * Each model should return a short response to the test prompt.
 * If a model fails, an error message is printed and the next model is tried.
```

```

*/
public class LLMSERVICE {

    /**
     * Main entry point to the application
     * @param args
     */
    public static void main(String[] args) {

        // Create an instance of LLMSERVICE to send prompts to the models
        LLMSERVICE service = new LLMSERVICE();

        // A simple one-sentence prompt used to verify each model is working.
        // Kept short so the test completes quickly.
        String testPrompt = "Name one common cooking ingredient. Reply in one sentence.";

        // List of all four model names used in this study.
        // These constants are defined in LLMSERVICE to avoid typos.
        String[] models = {
            LLMSERVICE.LLAMA,    // Meta Llama 3.2 3B

```

```
LLMService.PHI,      // Microsoft Phi-4-mini 3.8B
LLMService.QWEN,      // Alibaba Qwen 2.5 3B
LLMService.SEALION,   // AI Singapore Gemma-SEA-LION v4 4B
LLMService.MEDGEMMA   // Google MedGemma 4B (biomedical domain)
};
```

```
// Loop through each model, send the test prompt, and print the response.
```

```
// Each model is tested independently so a failure in one does not
```

```
// stop the remaining models from being tested.
```

```
for (String model : models) {
```

```
    System.out.println("-----");
```

```
    System.out.println("Model   : " + model);
```

```
    System.out.print("Response : ");
```

```
    try {
```

```
        // Send the prompt to the current model and print the response
```

```
        String response = service.prompt(model, testPrompt);
```

```
        System.out.println(response);
```

```
    } catch (Exception e) {
```

```
        // If the model fails (e.g. not loaded, timeout, server down),  
        // print the error and continue to the next model  
        System.out.println("ERROR - " + e.getMessage());  
    }  
}  
  
System.out.println("-----");  
System.out.println("Test complete.");  
}  
  
}
```

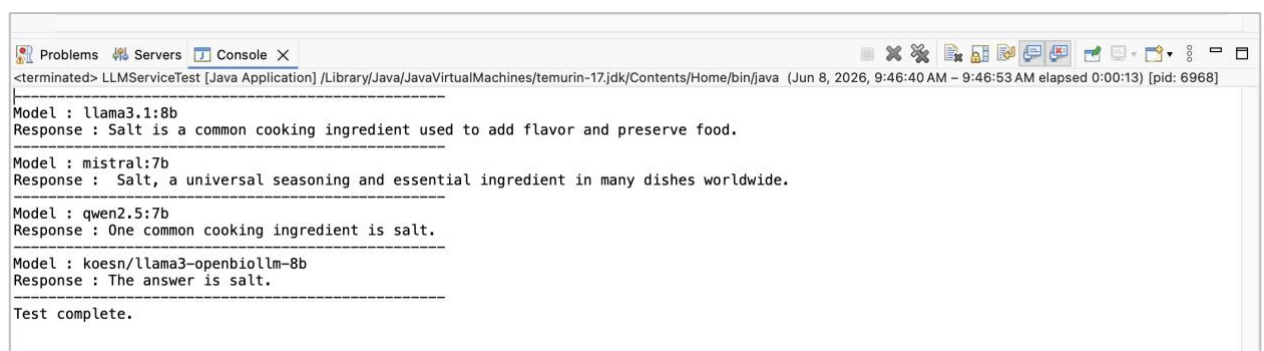
Figure 8: LLMServiceTest .java (updated)

5.4 Ensure Ollama is running

1. Before running, confirm the **Ollama** icon is visible in the **macOS** menu bar (MacOS) or **System Tray** (windows). If not, open **Ollama** from **Applications** (MacOS/Windows) first.

5.5 Run the test

1. Right-click LLMServiceTest.java in Package Explorer
2. Select Run As → Java Application
3. Watch the Console tab at the bottom of Eclipse. The output shall be similar as shown in Figure 9.



```
<terminated> LLMServiceTest [Java Application] /Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java (Jun 8, 2026, 9:46:40 AM - 9:46:53 AM elapsed 0:00:13) [pid: 6968]

Model : llama3.1:8b
Response : Salt is a common cooking ingredient used to add flavor and preserve food.

Model : mistral:7b
Response : Salt, a universal seasoning and essential ingredient in many dishes worldwide.

Model : qwen2.5:7b
Response : One common cooking ingredient is salt.

Model : koehn/llama3-openbiollm-8b
Response : The answer is salt.

Test complete.
```

Figure 9: Sample output from LLMServiceTest.java

5. Study the codes created from this section to enhance the understanding about the project.

6 Summary of Implementation

Step	Status
Step 1: Ollama Installation & LLM Pull	Done
Step 2: Project Configuration	Done
Step 3: Java Implementation	Done
Step 4: Nutritional analysis Implementation	Next

Document ends here
